
UNIVERSIDAD DE CANTABRIA

INGENIERÍA DEL SOFTWARE II

PRÁCTICA 5B

Alejandro López-Tormos Gervás

Date created:
Jueves, 30 de abril de 2026

PRUEBA DE ESTRUCTURAS DE DATOS

1.

Método get pruebas de caja negra

Parámetro	Clases de equivalencia válidas	Clases de equivalencias no válidas
Index	(1) [0 - lista.size())	(3) Negativo (4) [lista.size - inf]
Estado lista	(2) Lista con elementos	(5) Lista vacía

Clase de equivalencia	Valores
1	0, lista.size()/2, lista.size – 1
2	Lista con un elemento, lista con muchos elementos
3	-1, -200
4	lista.size(), 200
5	Lista vacía

Casos de prueba válidos	Casos de prueba no válidos
(0, lista con un elemento): 1	(-1, lista con un elemento): IndexOutOfBoundsException
(lista.size()/2, lista con muchos elementos): 4	(-200, lista con un elemento): IndexOutOfBoundsException
(lista.size – 1, lista con muchos elementos): 110	(lista.size(), lista con muchos elementos): IndexOutOfBoundsException
	(200, lista con muchos elementos): IndexOutOfBoundsException
	(1, lista vacía): IndexOutOfBoundsException

Método add pruebas de caja negra

Parámetro	Clases de equivalencia válidas	Clases de equivalencias no válidas
Elemento	(1) Elemento que está en el conjunto (2) Elemento que no está en el conjunto	(4) Null
Estado lista	(3) Estado actual de la lista	

Clase de equivalencia	Valores
1	Elemento que está en el conjunto
2	Elemento que no está en el conjunto
3	Lista con un elemento, lista con muchos elementos, lista vacía
4	null

Casos de prueba válidos	Casos de prueba no válidos
(Elemento que está en el conjunto, lista con un elemento): false	(null, lista con un elemento): NullPointerException
(Elemento que no está en el conjunto, lista con muchos elementos): true y elemento en la posición 2	
(elemento que no está en el conjunto, lista vacía): true y elemento en la posición 0	
(Elemento que no está en el conjunto, lista con un elemento): true y elemento en la posición 2	

Método remove pruebas de caja negra

Parámetro	Clases de equivalencia válidas	Clases de equivalencias no válidas
Index	(1) [0 - lista.size())	(3) Negativo (4) [lista.size - inf]
Estado lista	(2) Lista con elementos	(5) Lista vacía

Clase de equivalencia	Valores
1	0, lista.size()/2, lista.size – 1
2	Lista con un elemento, lista con muchos elementos
3	-1, -200
4	lista.size(), 200
5	Lista vacía

Casos de prueba válidos	Casos de prueba no válidos
(0, lista con un elemento): 1	(-1, lista con un elemento): IndexOutOfBoundsException
(lista.size()/2, lista con muchos elementos): 4	(-200, lista con un elemento): IndexOutOfBoundsException
(lista.size – 1, lista con muchos elementos): 110	(lista.size(), lista con muchos elementos): IndexOutOfBoundsException
	(200, lista con muchos elementos): IndexOutOfBoundsException
	(1, lista vacía): IndexOutOfBoundsException

Método size pruebas de caja negra

Parámetro	Clases de equivalencia válidas	Clases de equivalencias no válidas
Estado lista	(1) Estado de la lista	

Clase de equivalencia	Valores
1	Lista con un elemento, lista con muchos elementos, lista vacía

Casos de prueba válidos	Casos de prueba no válidos
(lista con un elemento): 1	
(lista con muchos elementos): 5	
(lista vacía): 0	

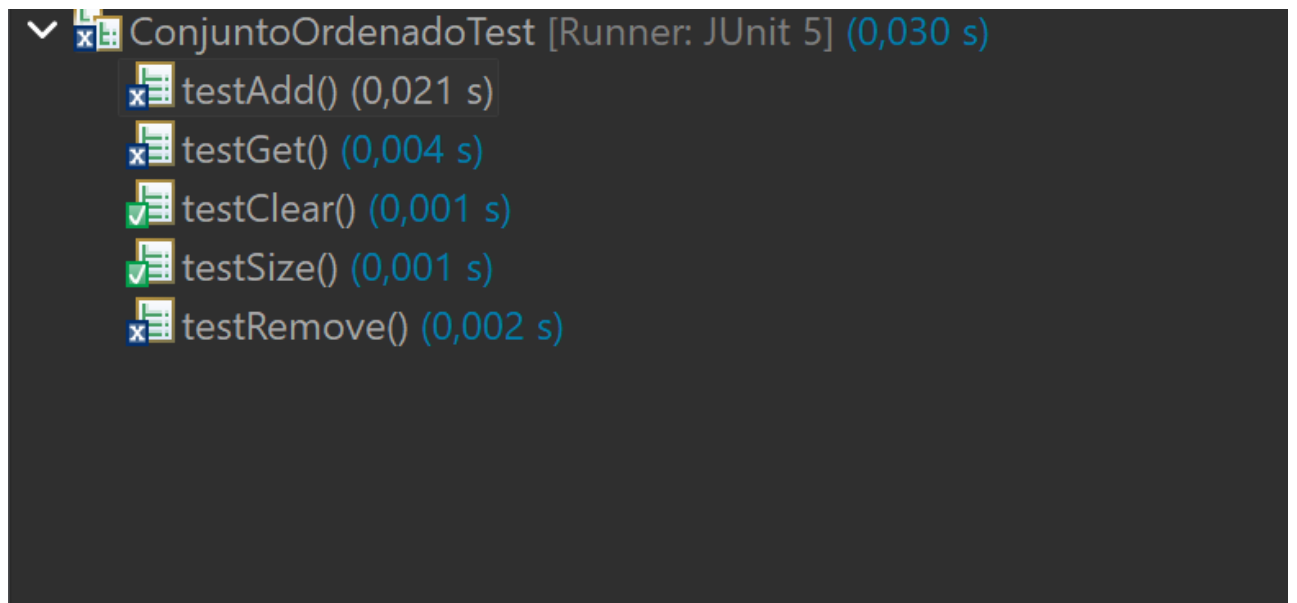
Método clear pruebas de caja negra

Parámetro	Clases de equivalencia válidas	Clases de equivalencias no válidas
Estado lista	(1) Estado de la lista	

Clase de equivalencia	Valores
1	Lista con un elemento, lista con muchos elementos, lista vacía

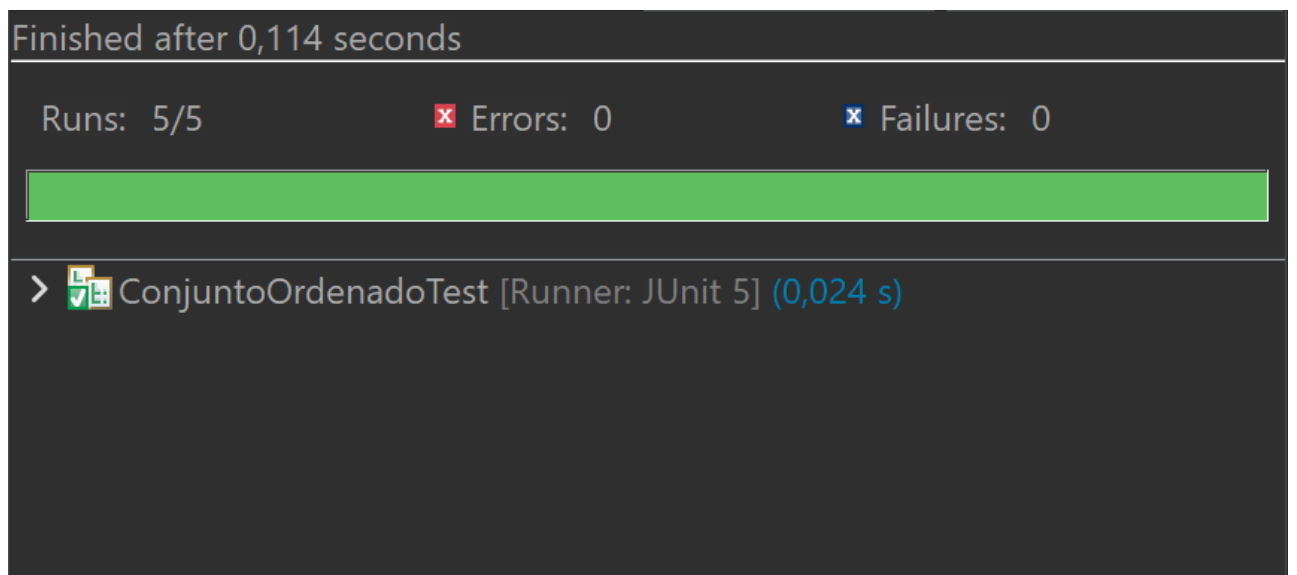
Casos de prueba válidos	Casos de prueba no válidos
(lista con un elemento): void (comprobar que size() da 0)	
(lista con muchos elementos): void (comprobar que size() da 0)	
(lista vacía): void (comprobar que size() da 0)	

Resultados de las pruebas de caja negra



Los resultados muestran como los tests generados por las pruebas de caja negra no pasan para la clase ConjuntoOrdenado. Se van a proponer algunos cambios para que la funcionalidad de esta clase sea la adecuada.

El primer error es cambiar la condición del while quitándolo y poniéndola dentro del bucle en forma de if. Si se encuentra un elemento igual directamente se devuelve false. Además, la lógica de la condición debería de ser igual y no menor que. Por último, se va a crear un if que maneje si un dato es mayor que el pasado por parámetro que indicará la posición de inserción.



▼  ConjuntoOrdenado.java	 100,0 %
>  ConjuntoOrdenado<E extends Comp	 100,0 %

Método get pruebas de caja blanca

Sentencia cubierta	Casos de prueba	Valor esperado
Indice	(0, lista con un elemento)	1

Método add pruebas de caja blanca

Cobertura de decisión:

Decisión	Caso true	Caso false
Elemento == null	(null): NullPointerException	(3, lista vacía): true
lista.size() != 0	(3, lista con elementos): true	(3, lista vacía): true
compareTo == 0	(1, lista con el elemento 1): false	(3, lista sin el elemento 3): true
compareTo < 0	(0, lista con varios elementos): true	(1000, lista con varios elementos): true

Cobertura de bucles

Decisión	Caso de prueba	Valor esperado
Indice < lista.size()	(3, lista vacía)	true
	(3, lista con un elemento)	true
	(3, lista con muchos elementos)	true

Método remove pruebas de caja blanca

Sentencia cubierta	Casos de prueba	Valor esperado
Indice	(0, lista con un elemento)	1

Método size pruebas de caja blanca

Sentencia cubierta	Casos de prueba	Valor esperado
Lista con muchos elementos	(lista con 5 elementos)	5

Método clear pruebas de caja blanca

Sentencia cubierta	Casos de prueba	Valor esperado
Lista con muchos elementos	(Lista con 5 elementos)	void, size() = 0

Resultaod de pruebas de caja blanca

Los tests de caja blanca pasan todos sin problema y se ve que el coverage es también del 100%.

